

TUGAS PRAKTIKUM 6

SISTEM OPERASI



MUHAMMAD MAULANA NUR SULAIMAN

2410651033

UNIVERITAS MUHAMMDIYAH JEMBER

FAKULTAS TEKNIK

PROGRAM STUDI TEKNIK INFORMATIKA

2025

Tugas 1: Analisis Deadlock

Jalankan program `deadlock_simple.c` dan jelaskan:

1. Mengapa program mengalami deadlock?
2. Kondisi deadlock apa saja yang terpenuhi?
3. Gambar diagram resource allocation graph-nya

```
alangenteng@LAPTOP-CV1QG677:~$ nano deadlock_simple.c
alangenteng@LAPTOP-CV1QG677:~$ gcc deadlock_simple.c -o deadlock_simple -lpthread
alangenteng@LAPTOP-CV1QG677:~$ ./deadlock_simple
=== SIMULASI DEADLOCK ===

Thread 1: Mencoba mengambil Lock 1...
Thread 1: Berhasil dapat Lock 1!
Thread 2: Mencoba mengambil Lock 2...
Thread 2: Berhasil dapat Lock 2!
Thread 1: Mencoba mengambil Lock 2...
Thread 2: Mencoba mengambil Lock 1...
^C
```

1. Mengapa program mengalami deadlock?

Program mengalami deadlock karena:

- Thread 1 mengunci Lock 1, lalu mencoba mengambil Lock 2.
- Thread 2 mengunci Lock 2, lalu mencoba mengambil Lock 1.

Keduanya menunggu kunci yang sedang dipegang thread lain, sehingga tidak ada yang bisa lanjut. Akibatnya, kedua thread berhenti selamanya.

Ilustrasi sederhana:

- Thread 1: "Gue punya kunci 1, mau ambil kunci 2"
- Thread 2: "Gue punya kunci 2, mau ambil kunci 1"
- Keduanya saling nunggu → macet → DEADLOCK.

2. Kondisi deadlock apa saja yang terpenuhi?

Deadlock terjadi jika 4 kondisi Coffman terpenuhi. Program ini memenuhi semuanya:

a) Mutual Exclusion

Setiap kunci (lock1 dan lock2) hanya bisa dipegang oleh *satu thread* pada satu waktu.

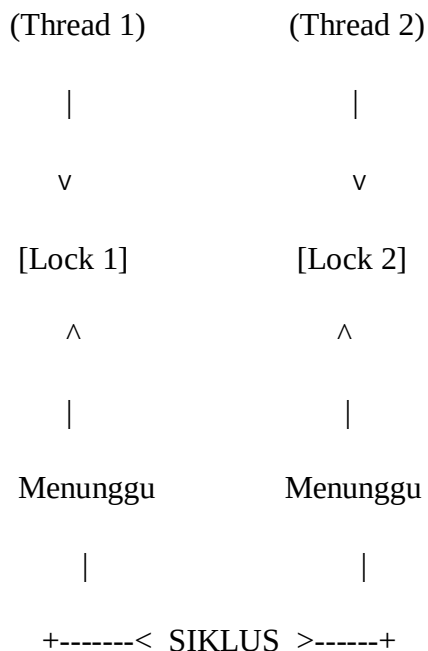
b) Hold and Wait

Thread memegang satu kunci dan menunggu kunci lain:

- Thread 1: pegang lock1, menunggu lock2

- Thread 2: pegang lock2, menunggu lock1
- c) No Preemption
Mutex tidak bisa direbut paksa. Harus dilepas secara sukarela oleh pemiliknya → ini tidak terjadi.
- d) Circular Wait
Ada lingkaran ketergantungan:
- Thread 1 → butuh Lock 2
 - Thread 2 → butuh Lock 1

3. Gambar diagram resource allocation graph-nya



Tugas 2: Implementasi Pencegahan

Modifikasi program deadlock_simple.c menggunakan teknik:

1. Lock ordering

Semua thread harus mengunci mutex dalam urutan yang sama.

Misalnya: Lock 1 → Lock 2 (selalu urutan itu).

Syntaxnya:

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
```

```
// Thread 1
```

```
void* thread1_func(void* arg) {
```

```
    printf("Thread 1: Mengambil Lock 1...\n");
```

```
    pthread_mutex_lock(&lock1);
```

```
    sleep(1); // simulasi kerja
```

```
    printf("Thread 1: Mengambil Lock 2...\n");
```

```
    pthread_mutex_lock(&lock2);
```

```
    printf("Thread 1: Selesai.\n");
```

```
    pthread_mutex_unlock(&lock2);
```

```
    pthread_mutex_unlock(&lock1);
```

```
    return NULL;
```

```
}
```

```
// Thread 2 (DIUBAH: lock 1 dulu, lalu lock 2)
```

```
void* thread2_func(void* arg) {
```

```
    printf("Thread 2: Mengambil Lock 1...\n");
```

```
    pthread_mutex_lock(&lock1);
```

```
    sleep(1); // simulasi kerja
```

```
    printf("Thread 2: Mengambil Lock 2...\n");
```

```
    pthread_mutex_lock(&lock2);
```

```
    printf("Thread 2: Selesai.\n");
```

```
    pthread_mutex_unlock(&lock2);
```

```
    pthread_mutex_unlock(&lock1);
```

```
    return NULL;
```

```
}
```

```
int main() {
```

```
    pthread_t t1, t2;
```

```

printf("=== Lock Ordering ===\n");

pthread_create(&t1, NULL, thread1_func, NULL);

pthread_create(&t2, NULL, thread2_func, NULL);


pthread_join(t1, NULL);

pthread_join(t2, NULL);


return 0;

}

```

Hasil

- **Tidak terjadi deadlock**
- Salah satu thread akan menunggu, tapi tetap selesai.

Hasil Output:

```

alanganteng@LAPTOP-CV1QG677:~$ nano deadlock_simple.c
alanganteng@LAPTOP-CV1QG677:~$ gcc deadlock_simple.c -o deadlock_simple -pthread
cc1: fatal error: deadlock_simple.c: No such file or directory
compilation terminated.
alanganteng@LAPTOP-CV1QG677:~$ gcc deadlock_simple.c -o deadlock_simple -pthread
alanganteng@LAPTOP-CV1QG677:~$ ./deadlock_simple
=== Lock Ordering ===
Thread 1: Mengambil Lock 1...
Thread 2: Mengambil Lock 1...
Thread 1: Mengambil Lock 2...
Thread 1: Selesai.
Thread 2: Mengambil Lock 2...
Thread 2: Selesai.

```

2. Try-lock dengan retry

Thread mencoba lock:

- Jika GAGAL → lepaskan lock lain yang sudah diambil, tunggu sebentar, dan coba lagi.

Ini mencegah circular wait.

Syntax nya:

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
```

```
void* thread1_func(void* arg) {
```

```
    while (1) {
```

```
        pthread_mutex_lock(&lock1);
```

```
        printf("Thread 1: Dapat Lock 1\n");
```

```
        sleep(1);
```

```
        if (pthread_mutex_trylock(&lock2) == 0) {
```

```
            printf("Thread 1: Dapat Lock 2\n");
```

```
            pthread_mutex_unlock(&lock2);
```

```
            pthread_mutex_unlock(&lock1);
```

```
            break;
```

```
        } else {
```

```
    printf("Thread 1: Gagal dapat Lock 2, retry...\n");

    pthread_mutex_unlock(&lock1);

    usleep(100000); // tunggu 0.1 detik
}

}

return NULL;
}
```

```
void* thread2_func(void* arg) {

    while (1) {

        pthread_mutex_lock(&lock2);

        printf("Thread 2: Dapat Lock 2\n");

        sleep(1);

        if (pthread_mutex_trylock(&lock1) == 0) {

            printf("Thread 2: Dapat Lock 1\n");

            pthread_mutex_unlock(&lock1);

            pthread_mutex_unlock(&lock2);

            break;

        } else {

            printf("Thread 2: Gagal dapat Lock 1, retry...\n");

            pthread_mutex_unlock(&lock2);


```



```

        usleep(100000);

    }

}

return NULL;
}

int main() {

    pthread_t t1, t2;

    printf("=== Try-Lock Retry ===\n");

    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}

```

Hasil

- **Tidak deadlock**
- Thread yang gagal lock akan **mundur (back off)** dan mencoba lagi.
- Lebih fleksibel untuk situasi yang kompleks.

Hasil Output:

```

=== Try-Lock Retry ===
Thread 1: Dapat Lock 1
Thread 2: Dapat Lock 2
Thread 1: Gagal dapat Lock 2, retry...
Thread 2: Dapat Lock 1
Thread 1: Dapat Lock 1
Thread 1: Dapat Lock 2

```

3. Bandingkan kedua metode tersebut

METODE	KELEBIHAN	KEKURANGAN
LOCK ORDERING	- Paling Sederhana - Garansi tidak Deadlock - Performa stabil	- Harus tahu semua urutan lock di awal - Tidak cocok jika lock banyak/ dinamis
Try-Lock + Retry	- Sangat fleksibel - Cocok untuk sistem besar dan lock kompleks - Tidak memblok thread	- Lebih kompleks - Ada busy-waiting, bisa boros CPU
EFISIENSI	Cepat dan stabil	Bisa lebih lambat karena retry

Tugas 3: Banker's Algorithm

Buat program baru dengan data:

- 4 proses
- 3 jenis resource
- Available: [5, 4, 3]
- Implementasikan request resource dan cek keamanan sistem

- Buat file program:
nano bankers.c

```

#include <stdio.h>
#include <stdbool.h>

#define P 4 // jumlah proses
#define R 3 // jenis resource

int available[R] = {5, 4, 3};

int max[P][R] = {
    {7, 5, 3},
    {3, 2, 2},
    {9, 0, 2},
    {2, 2, 2}
};

int allocation[P][R] = {
    {0, 1, 0},
    {2, 0, 0},
    {3, 0, 2},
    {2, 1, 1}
};

int need[P][R];

// Hitung matriks NEED
void calculateNeed() {
    for (int i = 0; i < P; i++)
        for (int j = 0; j < R; j++)
            need[i][j] = max[i][j] - allocation[i][j];
}

```

```
// Mengecek apakah system dalam safe state
```

```
bool isSafe() {
```

```
    int work[R];
```

```
    bool finish[P] = {false};
```

```
    for (int i = 0; i < R; i++)
```

```
        work[i] = available[i];
```

```
    int safeSeq[P];
```

```
    int count = 0;
```

```
    while (count < P) {
```

```
        bool found = false;
```

```
        for (int i = 0; i < P; i++) {
```

```
            if (!finish[i]) {
```

```
                bool canAllocate = true;
```

```
                for (int j = 0; j < R; j++)
```

```
                    if (need[i][j] > work[j]) {
```

```
                        canAllocate = false;
```

```
                        break;
```

```
                    }
```

```
            if (canAllocate) {
```

```
                for (int j = 0; j < R; j++)
```

```
                    work[j] += allocation[i][j];
```

```
                safeSeq[count++] = i;
```

```
                finish[i] = true;
```

```

        found = true;
    }
}

if (!found) {
    printf("\n ✖ Sistem TIDAK aman! Deadlock mungkin terjadi.\n");
    return false;
}

printf("\n ✔ Sistem dalam keadaan AMAN.\n");
printf("Safe sequence: ");
for (int i = 0; i < P; i++)
    printf("P%d ", safeSeq[i]);
printf("\n");
return true;
}

// Melakukan request resource
void requestResources(int process, int request[]) {
    printf("\n>>> Request P%d: [%d, %d, %d]\n",
        process, request[0], request[1], request[2]);

    // Validasi dengan NEED
    for (int j = 0; j < R; j++) {
        if (request[j] > need[process][j]) {
            printf(" ✖ ERROR: Request melebihi NEED!\n");
            return;
        }
    }
}

```

```

}

// Validasi dengan AVAILABLE
for (int j = 0; j < R; j++) {
    if (request[j] > available[j]) {
        printf(" ✕ Request ditolak: resource tidak cukup.\n");
        return;
    }
}

// Simulasi alokasi sementara
for (int j = 0; j < R; j++) {
    available[j] -= request[j];
    allocation[process][j] += request[j];
    need[process][j] -= request[j];
}

// Cek apakah aman
if (isSafe()) {
    printf(" 🟢 Request disetujui!\n");
} else {
    // rollback
    printf(" ⚠ Request ditolak, rollback dilakukan.\n");
    for (int j = 0; j < R; j++) {
        available[j] += request[j];
        allocation[process][j] -= request[j];
        need[process][j] += request[j];
    }
}
}

```

```

int main() {
    calculateNeed();
    isSafe();

    int req1[R] = {1, 0, 2};
    requestResources(1, req1);

    int req2[R] = {3, 3, 0};
    requestResources(0, req2);

    return 0;
}

```

Program ini mengimplementasikan **Banker's Algorithm** untuk 4 proses dan 3 jenis resource dengan Available = {5,4,3}. Fitur utama:

1. Menghitung matriks need = max - allocation.
 2. Mengecek apakah sistem **aman** (safe state) dan menampilkan **safe sequence** jika ada.
 3. Menerima **request** (permintaan) resource dari proses: memvalidasi terhadap need dan available, melakukan *tentative allocation*, lalu mengecek keamanan; jika tidak aman, rollback dilakukan.
- P = jumlah proses (4), R = jumlah jenis resource (3).
 - available = resource bebas saat ini.
 - max = kebutuhan maksimum setiap proses.
 - allocation = resource yang sudah dialokasikan ke setiap proses.

Hasil output:

```

alangenteng@LAPTOP-CV1QG677:~$ nano bankers.c
alangenteng@LAPTOP-CV1QG677:~$ gcc bankers.c -o bankers -pthread
alangenteng@LAPTOP-CV1QG677:~$ ./bankers

☑ Sistem dalam keadaan AMAN.
Safe sequence: P1 P2 P3 P0

>>> Request P1: [1, 0, 2]

☑ Sistem dalam keadaan AMAN.
Safe sequence: P1 P2 P3 P0
🔒 Request disetujui!

>>> Request P0: [3, 3, 0]

☑ Sistem dalam keadaan AMAN.
Safe sequence: P3 P1 P2 P0
🔒 Request disetujui!

```

Tugas 4: Kasus Nyata (Real Case)

Buat simulasi deadlock untuk kasus: "Dua orang ingin transfer uang antar rekening secara bersamaan"

- Orang A: transfer dari Rekening 1 ke Rekening 2
- Orang B: transfer dari Rekening 2 ke Rekening 1
- Implementasikan deadlock prevention

Dua orang melakukan transfer:

- Orang A: transfer dari Rekening 1 → Rekening 2
- Orang B: transfer dari Rekening 2 → Rekening 1

Jika keduanya mengunci rekening yang berbeda secara terbalik, deadlock bisa terjadi:

1. Orang A mengunci Rekening 1, menunggu Rekening 2
2. Orang B mengunci Rekening 2, menunggu Rekening 1
3. Kedua thread saling menunggu selamanya → DEADLOCK

- a. Bikin program deadlock tanpa pencegahan

```
nano deadlock.c
```

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```



```
pthread_mutex_t rekening1 = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_t rekening2 = PTHREAD_MUTEX_INITIALIZER;
```

```
void* transferA(void* arg) {
```

```
    printf("A: Mengunci Rekening 1...\n");
```

```
    pthread_mutex_lock(&rekening1);
```

```
    sleep(1); // Simulasi proses
```

```
    printf("A: Mencoba mengunci Rekening 2...\n");
```

```
    pthread_mutex_lock(&rekening2);
```

```
    printf("A: Transfer selesai!\n");
```

```
    pthread_mutex_unlock(&rekening2);
```

```
    pthread_mutex_unlock(&rekening1);
```

```
    return NULL;
```

```
}
```

```
void* transferB(void* arg) {
```

```
    printf("B: Mengunci Rekening 2...\n");
```

```
    pthread_mutex_lock(&rekening2);
```

```
sleep(1); // Simulasi proses

printf("B: Mencoba mengunci Rekening 1...\n");
pthread_mutex_lock(&rekening1);

printf("B: Transfer selesai!\n");

pthread_mutex_unlock(&rekening1);
pthread_mutex_unlock(&rekening2);
return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, transferA, NULL);
    pthread_create(&t2, NULL, transferB, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
```

```
}
```

Hasil:

Program akan **macet** (hang) karena kedua thread saling menunggu → DEADLOCK.

```
alangenteng@LAPTOP-CV1QG677:~$ nano deadlock.c
alangenteng@LAPTOP-CV1QG677:~$ gcc deadlock.c -o deadlock -pthread
alangenteng@LAPTOP-CV1QG677:~$ ./deadlock
A: Mengunci Rekening 1...
B: Mengunci Rekening 2...
B: Mencoba mengunci Rekening 1...
A: Mencoba mengunci Rekening 2...
^C
```

b. Deadlock Prevention (solusi)

nano deadlock_prevention.c

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
pthread_mutex_t rekening1 = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_t rekening2 = PTHREAD_MUTEX_INITIALIZER;
```

```
void lockInOrder(pthread_mutex_t* first, pthread_mutex_t* second) {
```

```
    pthread_mutex_lock(first);
```

```
    pthread_mutex_lock(second);
```

```
}
```

```
void unlockInOrder(pthread_mutex_t* first, pthread_mutex_t* second) {
```

```
    pthread_mutex_unlock(second);  
    pthread_mutex_unlock(first);  
}
```

```
void* transferA(void* arg) {  
    printf("A: Transfer R1 -> R2\n");  
  
    pthread_mutex_t* first = &rekening1;  
    pthread_mutex_t* second = &rekening2;  
  
    lockInOrder(first, second);  
  
    printf("A: Transfer berhasil!\n");  
  
    unlockInOrder(first, second);  
    return NULL;  
}
```

```
void* transferB(void* arg) {  
    printf("B: Transfer R2 -> R1\n");  
  
    pthread_mutex_t* first = &rekening1; // Ikuti urutan lock  
    pthread_mutex_t* second = &rekening2;
```

```

        lockInOrder(first, second);

        printf("B: Transfer berhasil!\n");

        unlockInOrder(first, second);

        return NULL;
    }

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, transferA, NULL);
    pthread_create(&t2, NULL, transferB, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}

```

Program ini mensimulasikan dua thread yang melakukan transfer antar dua rekening (R1 dan R2) dan mencegah deadlock dengan lock ordering — yaitu selalu mengunci rekening dengan urutan yang sama (rekening1 lalu rekening2) terlepas

dari arah transfer. Karena urutannya konsisten, kedua thread tidak akan saling menunggu satu sama lain → deadlock dicegah.

Dengan lock ordering:

- Semua thread mengunci rekening dalam urutan yang sama
- Tidak ada kasus 1 thread menunggu lock yang dipegang thread lain yang juga menunggu dia → Deadlock mustahil terjadi

Hasil output:

```
alangenteng@LAPTOP-CV1QG677:~$ nano deadlock_prevention.c
alangenteng@LAPTOP-CV1QG677:~$ gcc deadlock_prevention.c -o deadlock_prevention -pthread
alangenteng@LAPTOP-CV1QG677:~$ ./deadlock_prevention
A: Transfer R1 -> R2
A: Transfer berhasil!
B: Transfer R2 -> R1
B: Transfer berhasil!
```